Github（或者 Coding）账号：GlueNa2SiO3

个人博客密码学实验的链接：**https://github.com/Glue-Na2SiO3/GLUE**

实验题目（中文）：

现代密码学

实验一

1. Coursera Dan Boneh Week 1 Program Assignment
   Many Time Pad
2. PA1 option
   Write a program that allows you to "crack" ciphertexts generated using a Vigenere-like cipher, where byte-wise XOR is used instead of addition modulo 26.
3. http://www.cryptopals.com/sets/1
   (1) Convert hex to base64
   (2) Fixed XOR
   (3) Single-byte XOR cipher
   (4) Detect single-character XOR
   (5) Implement repeating-key XOR
   (6) Break repeating-key XOR
4. MTC3 Cracking SHA1-Hashed Passwords
   https://www.mysterytwisterc3.org/en/challenges/level-2/cracking-sha1-hashed-passwords

实验摘要（中文）：

题目描述（清楚描述题目中文，写出自己的理解，请勿复制原题目）

1 在一次性密码（One-Time Pad, OTP）中：一次性密码 (OTP) 的基础是**明文 M 与等长随机密钥 K 按字节异或 C = M ⊕ K 得到密文 C**。其安全性建立在密钥的**一次性使用**上。如果相同的密钥 K 被重复用于加密多个明文（即 Many-Time Pad），安全性将完全崩溃，因为攻击者可以通过密文间的关系直接推导出明文之间的关系。

2 重复密钥 XOR 加密（类似 Vigenère 密码的字节版）使用一个**重复的短密钥** K 对明文 M 进行异或加密 **（cipher[i] = plaintext[i] ⊕ key[i mod keylen]）**。在不知道密钥 K 的情况下，目标是**恢复出密钥和原始明文**。核心利用点是：**密钥的周期性重复**使得攻击者能结合统计学和排列组合方法，逐字节地恢复出密钥的每个组成部分。

3 这是一系列实现不同编码和 XOR 操作的基础功能，用于支撑后续的破解算法：

- **Hex/Base64 转换**：将十六进制字符串转换为字节，再编码为 Base64 字符串。
- **固定异或（Fixed XOR）**：对两个等长字节串进行逐字节异或操作，并输出结果的十六进制表示。
- **单字节 XOR 密码分析**：给定单字节密钥加密的密文，通过**穷举 0 到 255 的所有密钥**，并使用频率分析评分函数（寻找最接近英语文本的解）来确定正确的密钥和明文。
- **单字节 XOR 检测**：遍历一组密文，利用上述单字节 XOR 破解方法，找出其中最可能是用单字节 XOR 加密的密文。
- **实现重复密钥 XOR**：根据 **（cipher[i] = plaintext[i] ⊕ key[i mod keylen]）** 公式，实现加密功能。

4 **重复密钥 XOR 密码的破解流程**

在不预知密钥的情况下，利用统计学和频率分析恢复密钥和明文的完整流程如下：

- **猜测密钥长度（Keylen）**：通过计算不同长度下的归一化汉明距离，找出几个距离最小的作为候选密钥长度 k。
- **转置分组**：对于每个候选 k，将密文按步长 k 转置分组（即密文的第 i, i+k, i+2k, ... 个字节构成一组）。
- **单字节破解**：对每个转置后的分组独立应用第 3 点中的单字节 XOR 破解技术，以确定密钥 K 的对应字节。
- **最终解密与评估**：将所有破解出的单字节拼成完整的密钥 k，用 k 解密密文，并对最终明文进行整体评分（可结合密钥长度惩罚项），以选出最准确的解密结果。

过程（包括背景，原理：必要的公式，图表；步骤，如有必要画出流程图，给出主要实现步骤代码）

1.原理

利用密钥重复使用的漏洞，我们可以通过异或多个密文来推断出一些明文位置。

假设 C1 ⊕ C2 的某一位结果对应的明文字符是：

(1)两个字母异或 → 结果大多为非 ASCII 打印字符；

(2)字母与空格 (‘ ’)异或 → 结果大多仍是字母。

因此，如果 C1 ⊕ C2 的某一位结果在可打印范围内，可以猜测某个明文位置是空格。

通过统计某个位置在与多个其他密文 XOR 后仍是“可打印字符”的次数，就可以推测出哪一条密文在该位置是空格。

代码：

```python
def bytes_to_hex(b):
    return binascii.hexlify(b)

def xor_bytes(b1, b2):
    return bytes(a ^ b for a, b in zip(b1, b2))

ciphertexts_hex = [
    "315c4eeaa8b5f8aaf9174145bf43e1784b8fa00dc71d885a804e5ee9fa40b16349c146fb778cdf2d3aff021dfff5b403b510d0d0455468aeb98622b1
    "234c02ecbbfbafa3ed18510abd11fa724fcda2018a1a8342cf064bbde548b12b07df44ba7191d9606e4081ffde5ad46a5069d9f7f543bedb9c861bf2
    "32510ba9a7b2bba9b8005d43a304b5714cc0bb0c8a34884dd91304b8ad40b62b07df44ba6e9d8a2368e51d04e0e7b207b70b9b8261112bacb6c866a2
    "32510ba9aab2a8a4fd06414fb517b5605cc0aa0dc91a8908c2064ba8ad5ea06a029056f47a8ad3306ef5021eafe1ac01a81197847a5c68a1b78769a3
    "3f561ba9adb4b6ebec54424ba317b564418fac0dd35f8c08d31a1fe9e24fe56808c213f17c81d9607cee021dafe1e001b21ade877a5e68bea88d61b9
    "32510bfbacfbb9befd54415da243e1695ecabd58c519cd4bd2061bbde24eb76a19d84aba34d8de287be84d07e7e9a30ee714979c7e1123a8bd9822a3
    "32510bfbacfbb9befd54415da243e1695ecabd58c519cd4bd90f1fa6ea5ba47b01c909ba7696cf606ef40c04afe1ac0aa8148dd066592ded9f8774b5
    "315c4eeaa8b5f8bffd11155ea506b56041c6a00c8a08854dd21a4bbde54ce56801d943ba708b8a3574f40c00fff9e00fa1439fd0654327a3bfc860b5
    "271946f9bbb2aeadec111841a81abc300ecaa01bd8069d5cc91005e9fe4aad6e04d513e96d99de2569bc5e50eeeca709b50a8a987f4264edb6896fb5
    "466d06ece998b7a2fb1d464fed2ced7641ddaa3cc31c9941cf110abbf409ed39598005b3399ccfaf61d0315fca0a314be138a9f32503bedac8067f03
]

target_ciphertext_hex = "32510ba9babebbbefd001547a810e67149caee11d945cd7fc81a05e9f85aac650e9052ba6a8cd8257bf14d13e6f0a803b54

ciphertexts = [hex_to_bytes(c) for c in ciphertexts_hex]
target_ciphertext = hex_to_bytes(target_ciphertext_hex)
all_ciphertexts = ciphertexts + [target_ciphertext]

min_len = min(len(c) for c in all_ciphertexts)
max_len = max(len(c) for c in all_ciphertexts)

key = bytearray(max_len)
recovered_plaintext = bytearray(max_len)

for i in range(min_len):
```

```
(base) PS D:\GlueNa2SiO3\1experiment> D:/anaconda/Scripts/activate
(base) PS D:\GlueNa2SiO3\1experiment> conda activate base
(base) PS D:\GlueNa2SiO3\1experiment> & D:/anaconda/python.exe d:/GlueNa2SiO3/1experiment/solutions/danboneh_MTP.py
Recovered Key (first 83 bytes, hex): 66396e89c9dbd8cb9874352acd6395102eafce78aa7fed28a0766bc98d29c50b69b0339a19f8aa400c9c6d708f80c06bc77bfea71031489
ed8e802d25ba98732675daefcecd58a43106b268060804ed1089a24
Decrypted Message (first 83 bytes): The secuet message is: Whln using a strewm ciphhr4 9gve! usg th tkey myrO tcaQ NZc

Full Decrypted Message: The secuet message is: Whln using a strewm ciphhr4 9gve! usg th tkey myrO tcaQ NZc
```

运行结果：

Recovered Key (first 83 bytes, hex):

66396e89c9dbd8cb9874352acd6395102eafce78aa7fed28a0766bc98d29c50b69b0339a19f8aa4

00c9c6d708f80c06bc77bfea71031489ed8e802d25ba98732675daefcecd58a43106b268060804ed1089a24

Decrypted Message (first 83 bytes): The secuet message is: Whln using a strewm ciphhr4 9gve! usg th tkey myrO tcaQ NZc

Full Decrypted Message: The secuet message is: Whln using a strewm ciphhr4 9gve! usg th tkey myrO tcaQ NZc

2.背景： 一次性密码（One-Time Pad, OTP）在密钥只使用一次且与明文等长时是信息理论上不可破的（完全安全）。但题目里是重复使用同一短密钥（repeating-key XOR），即 cipher[i] = plain[i] ⊕ key[i mod k]。当同一密钥重复用于多条明文时，就不是 OTP，变成 Many-Time Pad / Vigenère-like XOR，会泄露统计信息，从而可被破解。

原理：

1. 猜密钥长度 k（key size）。

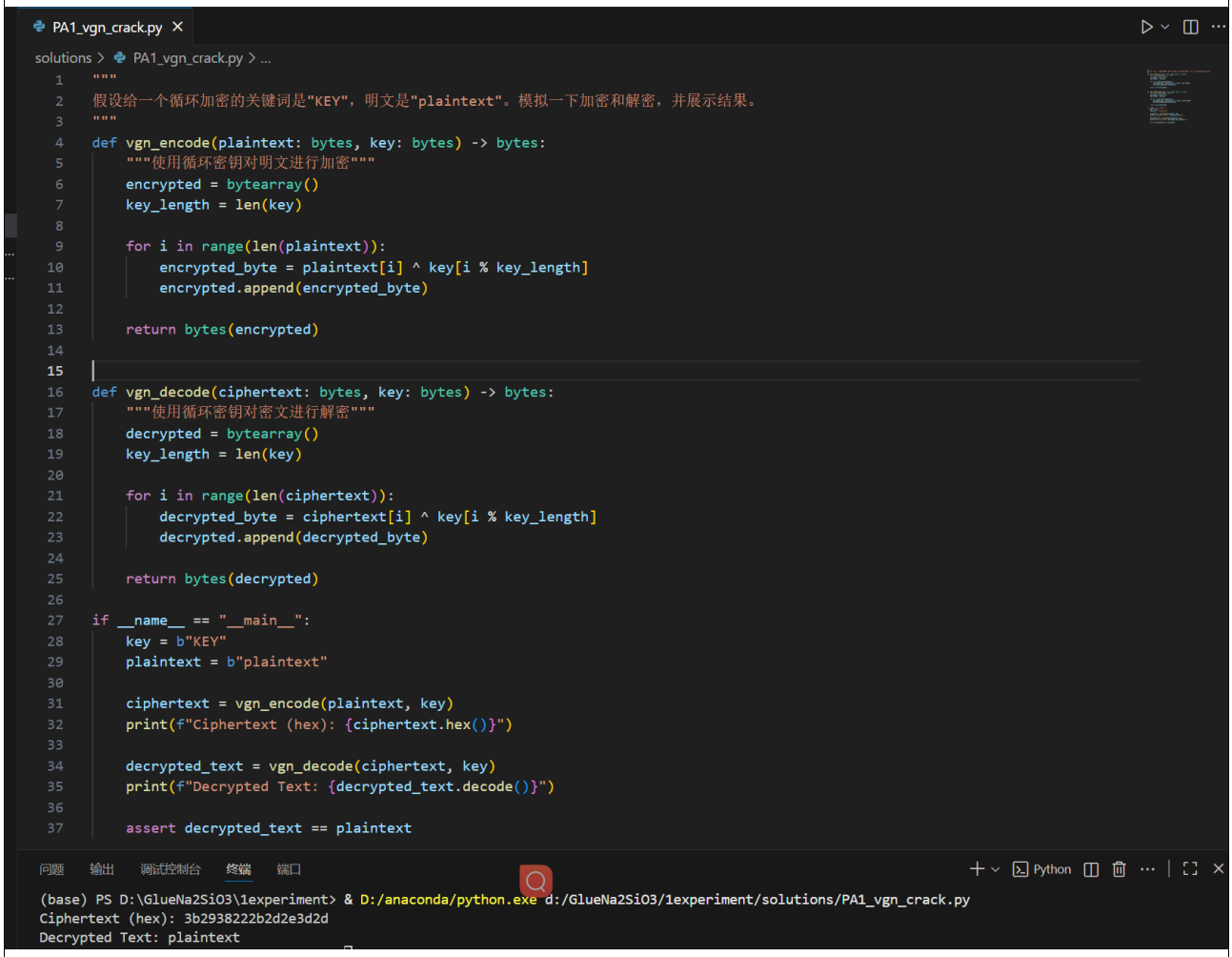
2.把密文按 k 转置：第 i 列由所有位置 p 满足 p % k == i 的字节组成。每列相当于“用单字节密钥”重复 XOR 的序列。

3.对每列单独破解单字节密钥（单字节 XOR 破解），用频率/评分选择最可能的单字节。

4.把各列的单字节拼成完整密钥，解出整段明文。

人工微调或使用更强的评分函数（n-gram / log-likelihood）来验证/修正结果。

代码及输出结果：



```python
"""
假设给一个循环加密的关键词是"KEY"，明文是"plaintext"。模拟一下加密和解密，并展示结果。
"""
def vgn_encode(plaintext: bytes, key: bytes) -> bytes:
    """使用循环密钥对明文进行加密"""
    encrypted = bytearray()
    key_length = len(key)

    for i in range(len(plaintext)):
        encrypted_byte = plaintext[i] ^ key[i % key_length]
        encrypted.append(encrypted_byte)

    return bytes(encrypted)


def vgn_decode(ciphertext: bytes, key: bytes) -> bytes:
    """使用循环密钥对密文进行解密"""
    decrypted = bytearray()
    key_length = len(key)

    for i in range(len(ciphertext)):
        decrypted_byte = ciphertext[i] ^ key[i % key_length]
        decrypted.append(decrypted_byte)

    return bytes(decrypted)

if __name__ == "__main__":
    key = b"KEY"
    plaintext = b"plaintext"

    ciphertext = vgn_encode(plaintext, key)
    print(f"Ciphertext (hex): {ciphertext.hex()}")

    decrypted_text = vgn_decode(ciphertext, key)
    print(f"Decrypted Text: {decrypted_text.decode()}")

    assert decrypted_text == plaintext
```

```
(base) PS D:\GlueNa2SiO3\1experiment> & D:/anaconda/python.exe d:/GlueNa2SiO3/1experiment/solutions/PA1_vgn_crack.py
Ciphertext (hex): 3b2938222b2d2e3d2d
Decrypted Text: plaintext
```

3.原理：

编码/解码

hex 字符串 → bytes：bytes.fromhex(hexstr)

bytes → base64：base64.b64encode(bytes)

base64 → bytes：base64.b64decode(str)

XOR

字节异或（字节数组逐字节异或）：c[i] = a[i] ^ b[i]

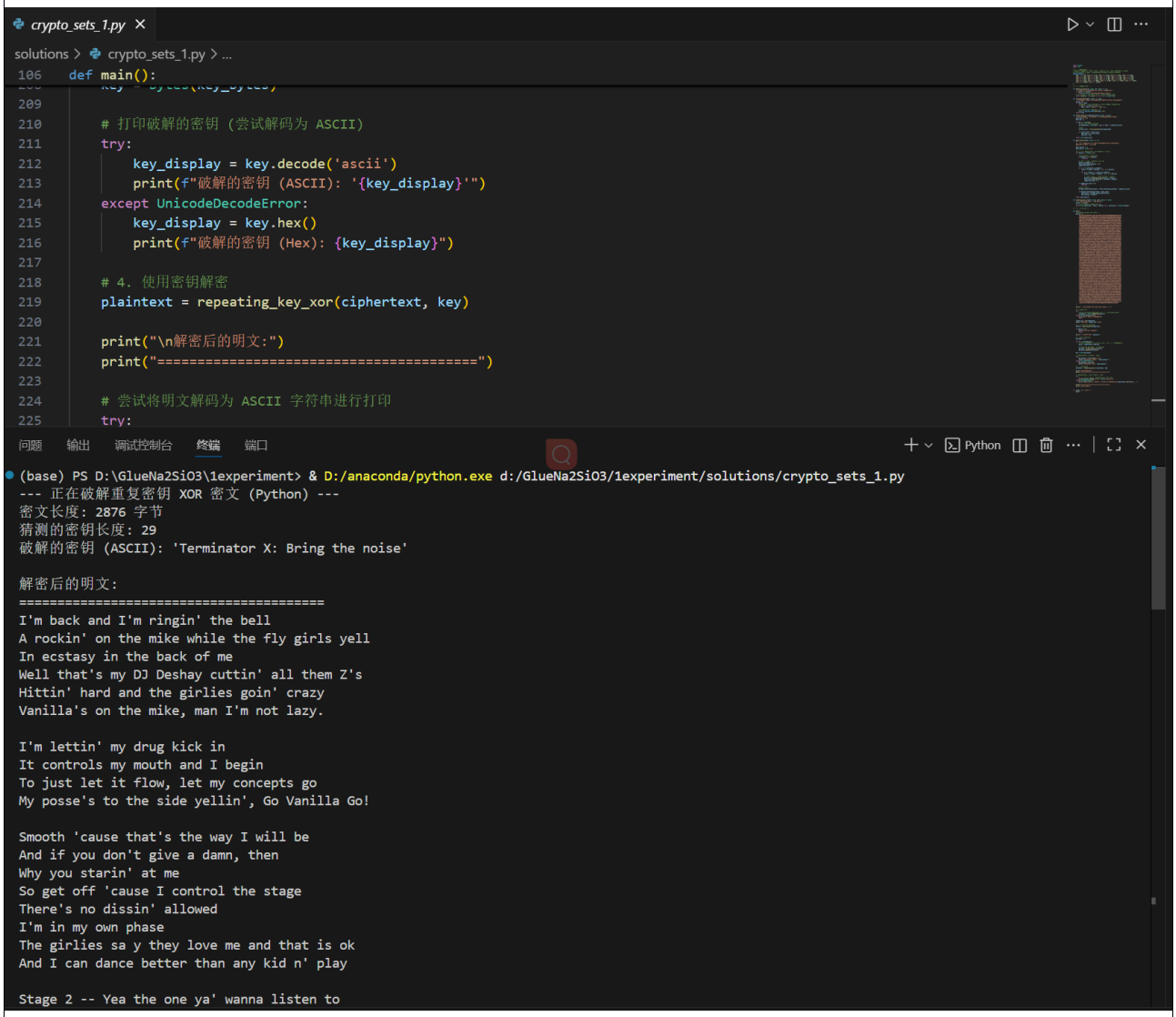
单字节 XOR：对每个字节与同一个 0..255 的值异或。

汉明距离（Hamming distance）

两字节串按比特位比较，不同位数之和。

计算：对每字节 x ^ y，然后统计该字节 1 的个数（v.bit_count()）。

代码及输出结果：



4. 背景

MysteryTwister C3 的这类题目是密码破解 / 密码学练习：给出一组 SHA-1 哈希值（通常是密码经 SHA-1 处理后的十六进制字符串），目标是找出原始明文密码。这是典型的离线密码破解任务（attacker 已有 hash 值，可以在本地做任意次尝试）。结合题目所给的罗列，我们可以分析把所有可能的字符放在那里，最后只取 8 位排列组合，进行 hash 计算，判断哪个是否合适。

哈希函数：H = SHA1(M)，把任意长度消息 M 映射到固定 160-bit（20 字节）的摘要 H。

单向：理论上无法有效从 H 计算出 M（但短弱密码可以通过穷举找到）。

抗碰撞性：找到两个不同 M1 != M2 使 SHA1(M1)=SHA1(M2) 的难度很高（但 SHA-1 已被实践性攻击弱化，不再推荐做安全用途）。



由于思路较难，更换解法，把密码中可能出现的字母写出来然后再暴力破解代码和输出：

总结（完成心得与其它，主要自己碰到的问题和解决问题的方法）

这次学习不仅加深了我对 XOR 运算在密码学中作用的理解，更重要的是提供了一个完整的思维框架：如何从密文的统计特性中寻找突破口，并将一个复杂的大问题系统地拆解为可控的小问题。重复密钥 XOR 密码的破解过程，是现代密码分析的绝佳入门案例，它要求分析者不仅要掌握数学运算，更要理解数据编码、语言统计等多个维度的知识。

参考文献（包括参考的书籍，论文，URL 等，很重要）